

GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

IAS QUE COOPERAM: COLABORAÇÃO ENTRE AGENTES E MODELOS GENERATIVOS



6

LISTA DE FIGURAS

Figura 1 - Single Agent <i>versus</i> Multi-Agent	7
Figura 2 - Diferença entre LLM e um Agente de IA	14
Figura 3 - LLM: o ciclo percepção – planejamento – ação	15

EMANDA

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de divisão de tarefas de IA	8
Código-fonte 2 - Exemplo de agentes especializados.....	9
Código-fonte 3 - Criando agente com o Agents SDK	17
Código-fonte 4 - Colaboração entre agentes com o Agents SDK	18
Código-fonte 5 - Pseudo-código: Handoffs Agents SDK	22
Código-fonte 6 - Agente simples com tool customizada.....	26
Código-fonte 7 - Agente simples com tool built-in de Web Search.....	27
Código-fonte 8 - Agente com handoffs.....	30
Código-fonte 9 - Agente com guardrails.....	36
Código-fonte 10 - Agente com memória.....	39

SUMÁRIO

1 IAS QUE COOPERAM: COLABORAÇÃO ENTRE AGENTES E MODELOS GENERATIVOS	6
1.1 A Nova Era da IA Colaborativa.....	6
2 O QUE É UM SISTEMA MULTIAGENTE?	11
3 AGENTES INTELIGENTES NA ERA DOS LLMS	13
4 AGENTES COM AGENTS SDK DA OPENAI	16
5 ARQUITETURA E CONCEITOS BÁSICOS DO OPENAI AGENTS SDK	20
5.1 Os Quatro Pilares Fundamentais	20
5.2 O Ciclo de Execução de um Agente (Loop Interno)	21
5.3 Delegações (Handoffs) como Componentes Nativos	22
5.4 Guardrails.....	22
5.5 Sessões e Histórico Contextual.....	23
5.6 Observabilidade via Tracing	23
6 FERRAMENTAS (FUNCTION TOOLS)	25
6.1 O que são Tools no Agents SDK.....	25
6.2 Definindo uma Function Tool.....	26
6.3 Uso de Hosted Tools	27
6.4 Agentes como Ferramentas (Agents as Tools)	27
6.5 Desafios e Boas Práticas	28
7 HANDOFFS: DELEGANDO CONTROLE ENTRE AGENTES	29
7.1 O conceito de handoff no SDK	29
7.2 Como criar um handoff	29
7.3 Personalizações e controle de handoffs.....	30
7.4 Casos de uso	31
7.4.1 Handoff de agente triador a especialistas de atendimento.....	31
7.5 Retorno para agente triador (handoff bidirecional)	31
7.6 Boas práticas.....	32
8 GUARDRAILS: VALIDANDO ENTRADA E SAÍDA COM SEGURANÇA	33
8.1 O papel dos Guardrails.....	33
8.2 Funcionamento técnico de Guardrails	34
8.2.1 Input Guardrails	34
8.2.2 Output Guardrails	34
8.3 Guardrails em ação	34
8.4 Boas práticas.....	36
8.5 Desafios e Limitações	36
9 SESSÕES E HISTÓRICO DE CONVERSAÇÃO	38
9.1 O que é uma Sessão no Agents SDK	38
9.1.1 Como funciona internamente	38
9.1.2 Uso de Sessão	39
9.1.3 Gerenciamento e limitações do histórico de memória da Session	40
9.1.4 Sessões em agentes de tempo real (RealtimeSession)	40
9.1.5 Importância prática	40

10 COMPARAÇÕES COM OUTROS FRAMEWORKS, LIMITAÇÕES E BOAS PRÁTICAS	42
10.1 Comparações com outras abordagens e frameworks	42
10.2 Limitações técnicas conhecidas	43
10.3 Boas práticas de uso	43
CONCLUSÃO.....	45
REFERÊNCIAS.....	47

EMENDAS

1 IAS QUE COOPERAM: COLABORAÇÃO ENTRE AGENTES E MODELOS GENERATIVOS

1.1 A Nova Era da IA Colaborativa

Durante décadas, a evolução da Inteligência Artificial foi guiada por uma ideia implícita: construir modelos cada vez mais poderosos capazes de resolver problemas sozinhos. Desde sistemas especialistas até redes neurais profundas e, mais recentemente, grandes modelos de linguagem, o paradigma dominante sempre foi o da inteligência centralizada. Nesse modelo, um único sistema recebe uma entrada, processa a informação e produz uma resposta. Em muitos cenários, essa abordagem continua sendo extremamente eficaz. No entanto, à medida que os problemas enfrentados pela sociedade se tornam mais complexos, dinâmicos e interdependentes, começa a surgir uma limitação estrutural nesse paradigma.

A realidade é que muitos desafios modernos não são apenas difíceis do ponto de vista computacional, mas também organizacional. Eles exigem múltiplas formas de raciocínio, diferentes especializações e capacidade de adaptação contínua. Sistemas monolíticos tendem a apresentar gargalos quando precisam lidar com múltiplos objetivos, diferentes fontes de informação e decisões simultâneas. É justamente nesse ponto que surge a necessidade de um novo modelo de inteligência: um **modelo baseado na colaboração**.

Os sistemas multiagentes representam essa mudança de paradigma. Em vez de depender de uma única entidade inteligente, eles são compostos por múltiplos agentes que interagem entre si em um ambiente compartilhado. Cada agente possui autonomia, capacidade de decisão e um papel específico dentro do sistema. A inteligência global emerge da interação entre essas entidades, e não da capacidade isolada de uma única delas. Em termos formais, um sistema multiagente pode ser definido como um sistema computacional composto por múltiplos agentes que interagem para alcançar objetivos individuais ou coletivos.

Essa abordagem se inspira diretamente em sistemas naturais e sociais. Pense em uma equipe médica em um hospital, em uma cadeia logística global ou em um time de engenharia trabalhando em um projeto complexo. Nenhum indivíduo domina

todo o conhecimento necessário, mas a colaboração permite que o grupo resolva problemas que seriam impossíveis para uma única pessoa. De maneira semelhante, sistemas multiagentes distribuem tarefas e responsabilidades entre diferentes componentes, permitindo que o conjunto resolva problemas de alta complexidade.

Com o avanço dos modelos de linguagem de larga escala, essa ideia ganha ainda mais força. Hoje, um agente pode ser implementado como um sistema baseado em LLM capaz de compreender linguagem natural, raciocinar sobre tarefas e decidir quais ações executar. Em um sistema multiagente, esses agentes podem assumir papéis especializados e trabalhar em conjunto para atingir um objetivo maior.

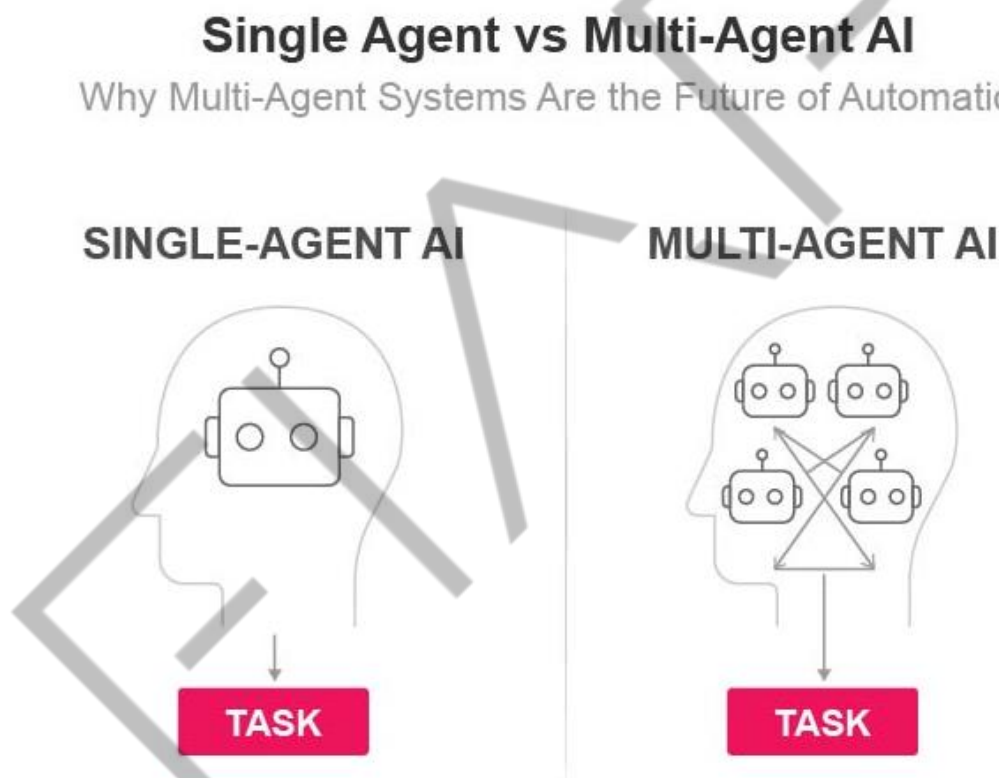


Figura 1 - Single Agent versus Multi-Agent

Fonte: Medium: Single Agent vs Multi-Agent AI: Why Multi-Agent Systems Are the Future of Automation (2025), adaptado por FIAP (2026)

Para entender essa transição de forma mais concreta, podemos construir um exemplo simples em Python. Para executar este exemplo, primeiro iremos criar uma chave de API para utilizarmos modelos da OpenAI. Para isso, basta criar um login no endereço <<https://platform.openai.com/>>. Após isso, na tela inicial, vá em API Keys (na aba lateral esquerda) e, em seguida, clique no botão “Create new secret key”. Copie a chave gerada e insira na aba “Secrets” do seu notebook no Google Colab, onde o nome do secret deverá ser OPENAI_API_KEY e o valor será a chave copiada

no OpenAI Platform. Após esses passos, você poderá utilizar modelos da OpenAI, através de sua API oficial, diretamente no Google Colab.

Suponha que queremos criar um sistema capaz de analisar um relatório e gerar um resumo com recomendações. Em um modelo tradicional, um único sistema faria todo o trabalho. Em um sistema multiagente, podemos dividir o problema entre agentes especializados.

```

1 | from openai import OpenAI
2 | from google.colab import userdata
3 | from os import environ
4 |
5 | environ['OPENAI_API_KEY'] =
  userdata.get('OPENAI_API_KEY')
6 |
7 | client = OpenAI()
8 |
9 | def agente_leitor(texto):
10 |     prompt = f"Leia o seguinte relatório e extraia os
  pontos principais:\n\n{texto}"
11 |     resposta = client.responses.create(
12 |         model="gpt-4o-mini",
13 |         input=prompt
14 |     )
15 |     return resposta.output_text
16 |
17 | def agente_analista(resumo):
18 |     prompt = f"Com base nos pontos principais abaixo,
  gere recomendações estratégicas:\n\n{resumo}"
19 |     resposta = client.responses.create(
20 |         model="gpt-4o-mini",
21 |         input=prompt
22 |     )
23 |     return resposta.output_text
24 |
25 | relatorio = "As vendas caíram 12% no último trimestre
  devido à ruptura de estoque e baixa visibilidade
  promocional."
26 |
27 | pontos = agente_leitor(relatorio)
28 | recomendacoes = agente_analista(pontos)
29 |
30 | print(recomendacoes)

```

Código-fonte 1 - Exemplo de divisão de tarefas de IA

Fonte: Elaborado pelo autor (2026)

Nesse pequeno exemplo, já podemos observar a essência da colaboração. O primeiro agente é responsável pela compreensão do texto, enquanto o segundo

assume o papel de tomada de decisão estratégica. Separadamente, cada agente resolve um subproblema. Em conjunto, eles produzem uma solução mais estruturada.

Outro aspecto importante dessa nova era é que a colaboração não ocorre apenas entre agentes idênticos. Diferentes modelos podem coexistir dentro de um mesmo sistema. Um agente pode ser especializado em linguagem, outro em visão computacional, outro em previsão temporal. O resultado é um ecossistema de inteligência distribuída.

Podemos ilustrar essa ideia com um exemplo que combina dois agentes distintos: um responsável por interpretar dados numéricos e outro por gerar explicações em linguagem natural.

```

1 | import pandas as pd
2 |
3 | dados = pd.DataFrame({
4 |     "mes": ["Jan", "Fev", "Mar"],
5 |     "vendas": [120, 95, 140]
6 | })
7 |
8 | def agente_dados(df):
9 |     return df.describe().to_string()
10 |
11 | def agente_explicador(analise):
12 |     prompt = f"Explique os seguintes indicadores de
vendas para um gestor:\n\n{analise}"
13 |     resposta = client.responses.create(
14 |         model="gpt-4o-mini",
15 |         input=prompt
16 |     )
17 |     return resposta.output_text
18 |
19 | analise = agente_dados(dados)
20 | explicacao = agente_explicador(analise)
21 |
22 | print(explicacao)
23 |

```

Código-fonte 2 - Exemplo de agentes especializados
Fonte: Elaborado pelo autor (2026)

Esse tipo de divisão de responsabilidades reflete a lógica central dos sistemas multiagentes: problemas complexos podem ser decompostos em subtarefas executadas por entidades autônomas que interagem entre si. Essa decomposição não apenas melhora a eficiência, mas também aumenta a robustez do sistema, pois falhas podem ser isoladas e tratadas localmente.

O ponto central é que estamos presenciando uma mudança fundamental na forma como construímos Inteligência Artificial. Em vez de buscar modelos cada vez maiores e mais abrangentes, passamos a projetar ecossistemas compostos por agentes especializados que colaboram entre si. Essa abordagem não apenas amplia a capacidade de resolução de problemas, mas também aproxima os sistemas artificiais da maneira como a inteligência coletiva humana opera.

Essa nova era da IA colaborativa estabelece as bases para o restante do capítulo, no qual exploraremos como esses agentes são construídos, como interagem e como podem ser organizados em arquiteturas capazes de executar workflows complexos de forma autônoma.

2 O QUE É UM SISTEMA MULTIAGENTE?

Para compreender profundamente o funcionamento da inteligência colaborativa, é necessário formalizar o conceito de sistema multiagente. Um sistema multiagente pode ser entendido como um sistema computacional composto por múltiplas entidades inteligentes, chamadas agentes, que interagem entre si dentro de um ambiente compartilhado com o objetivo de alcançar metas individuais ou coletivas. Diferentemente de sistemas tradicionais baseados em uma única entidade que toma decisões, esses sistemas distribuem percepção, raciocínio e ação entre múltiplos componentes autônomos.

Essa distribuição não é apenas uma escolha arquitetural, mas uma necessidade prática. Muitos problemas contemporâneos são naturalmente distribuídos, dinâmicos e incertos. Um sistema de logística global, por exemplo, envolve diferentes fontes de informação, decisões simultâneas e múltiplas restrições. Um único modelo dificilmente consegue manter uma visão completa e atualizada de todo o sistema. Em contraste, em um sistema multiagente, cada agente possui uma visão local do ambiente e atua com base nessa percepção limitada. A inteligência global emerge da interação entre essas visões parciais.

Um aspecto central dessa definição é a autonomia. Cada agente é capaz de perceber seu ambiente, tomar decisões e executar ações sem depender de um controle centralizado. Isso não significa que os agentes operam de forma isolada. Pelo contrário, eles se comunicam, negociam e coordenam suas ações para atingir objetivos comuns ou compatibilizar interesses divergentes. Esse comportamento coletivo pode ser cooperativo, quando todos trabalham para um mesmo objetivo, ou competitivo, quando existem metas conflitantes.

Podemos construir uma analogia com um sistema de controle de tráfego aéreo. Cada avião pode ser interpretado como um agente com objetivos próprios, como chegar ao destino com segurança e eficiência. Ao mesmo tempo, existe a necessidade de coordenação para evitar colisões e otimizar rotas. O resultado final depende das decisões individuais e da interação entre todos os participantes do sistema.

Outro conceito fundamental é o ambiente. Um sistema multiagente sempre opera dentro de um contexto no qual os agentes percebem informações e realizam ações. Esse ambiente pode ser virtual, como um sistema de recomendação de um e-commerce, ou físico, como um conjunto de robôs colaborativos. A interação com o ambiente influencia diretamente o comportamento dos agentes e pode introduzir incertezas que exigem adaptação contínua.

Em síntese, um sistema multiagente é caracterizado pela presença de múltiplos agentes autônomos que interagem em um ambiente comum para resolver problemas que excedem a capacidade de um único agente. Essa abordagem amplia a escalabilidade dos sistemas de IA e aumenta sua robustez e adaptabilidade.

Essa definição estabelece a base conceitual necessária para avançarmos para o entendimento de como esses agentes são implementados na prática, especialmente no contexto dos modelos de linguagem modernos que passaram a atuar como motores cognitivos dessas entidades inteligentes.

3 AGENTES INTELIGENTES NA ERA DOS LLMS

Para compreender o funcionamento dos sistemas multiagentes modernos, é essencial entender primeiro o papel dos agentes inteligentes na era dos grandes modelos de linguagem. Tradicionalmente, agentes em Inteligência Artificial eram construídos com regras fixas, modelos probabilísticos ou algoritmos de otimização. Esses agentes eram eficazes em ambientes bem definidos, mas apresentavam limitações significativas quando expostos a contextos ambíguos, dinâmicos ou ricos em linguagem natural.

A chegada dos Large Language Models (LLMs) transformou profundamente esse cenário. Hoje, um agente inteligente pode ser construído a partir de um modelo de linguagem que atua como o núcleo cognitivo do sistema. Nesse contexto, o LLM funciona como o “cérebro” do agente, responsável por compreender instruções, decompor problemas e planejar ações com base em objetivos definidos.

No entanto, um modelo de linguagem isolado não é, por si só, um agente. Ele é essencialmente um sistema que transforma texto de entrada em texto de saída. Para que ele se torne um agente, é necessário integrá-lo a uma arquitetura que permita perceber o ambiente, tomar decisões e executar ações. Esse processo transforma um modelo passivo em uma entidade ativa capaz de operar em ciclos contínuos de percepção, raciocínio e execução.

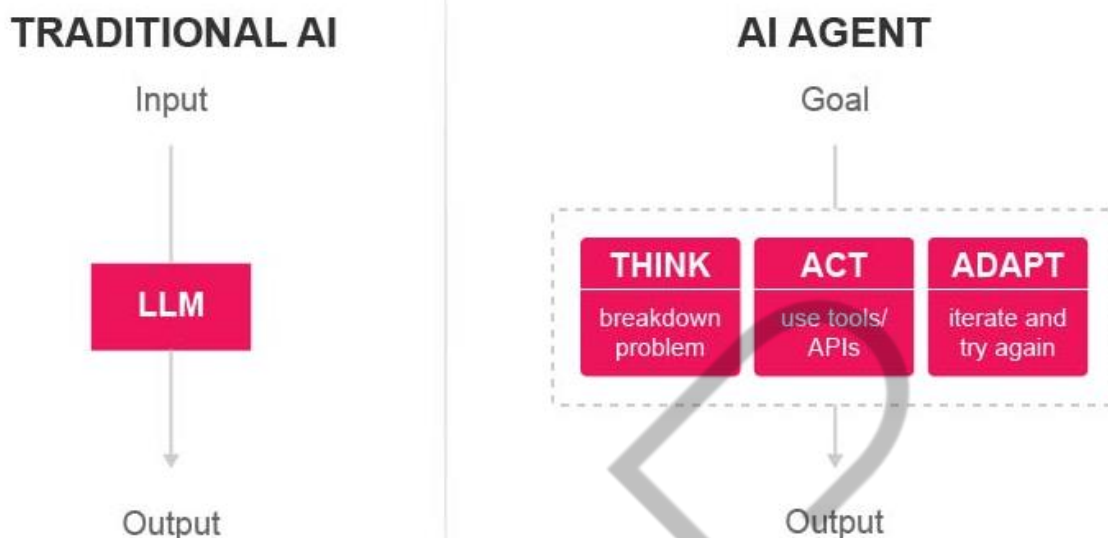


Figura 2 - Diferença entre LLM e um Agente de IA
Fonte: Tutorials DOJO: What Are AI Agents? (2025), adaptado por FIAP (2026)

Essa arquitetura pode ser compreendida por meio de três componentes fundamentais. O primeiro é a percepção, que corresponde à capacidade do agente de coletar informações do ambiente. Essas informações podem vir de texto, imagens, sensores ou bancos de dados. O segundo é o raciocínio, que ocorre dentro do modelo de linguagem e permite interpretar o contexto e decidir quais ações são necessárias. O terceiro é a ação, que consiste na execução de tarefas no ambiente por meio de ferramentas ou sistemas externos.

Essa estrutura é frequentemente descrita como o ciclo percepção, planejamento, ação, que define o funcionamento de agentes inteligentes modernos. Esse ciclo permite que o agente responda a comandos e formule estratégias para alcançar objetivos complexos.

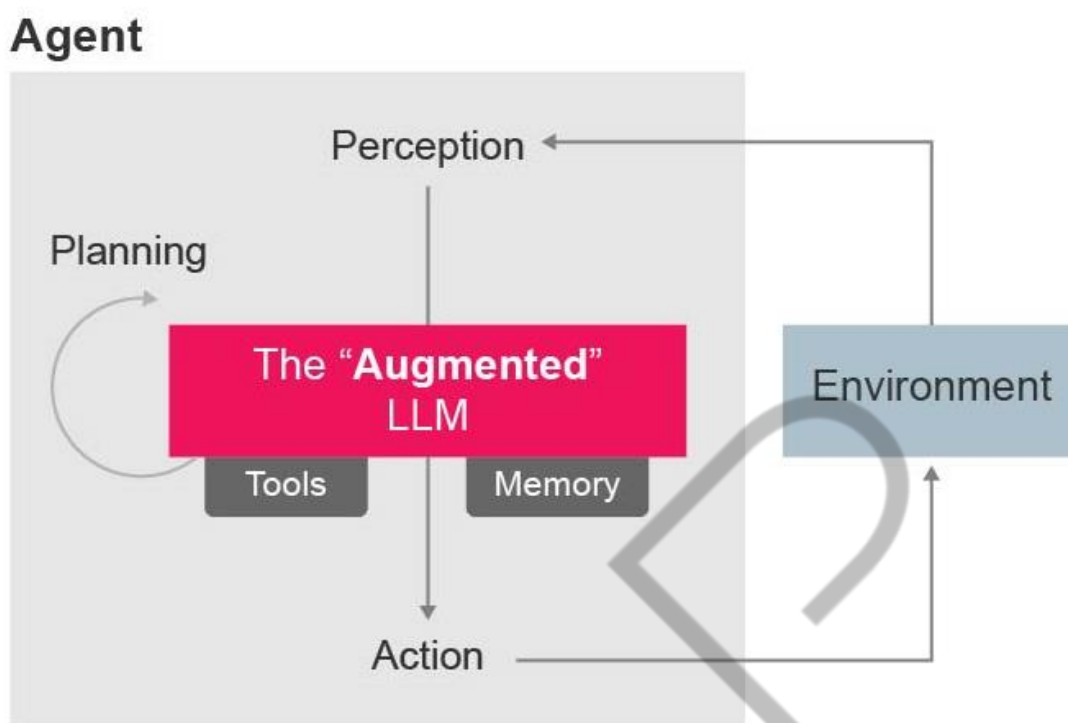


Figura 3 - LLM: o ciclo percepção – planejamento – ação

Fonte: Maarten Grootendorst: A Visual Guide to LLM Agents (2025), adaptado por FIAP (2026)

A capacidade de planejar e executar múltiplas etapas é uma das principais características dos agentes baseados em LLMs. Diferentemente de sistemas tradicionais, que seguem fluxos rígidos, esses agentes podem adaptar seu comportamento com base no contexto e nas informações disponíveis.

Outra característica relevante é que agentes modernos não operam apenas em um único ciclo. Eles podem executar múltiplas iterações de percepção e ação, refinando suas decisões ao longo do tempo. Isso permite que lidem com ambientes incertos e tarefas abertas.

Esse tipo de estrutura demonstra como agentes podem evoluir de respostas pontuais para sistemas que planejam sequências de ações.

A combinação entre percepção, raciocínio e execução representa a base da inteligência agentiva contemporânea. Essa arquitetura modular permite que agentes sejam integrados em sistemas maiores e, posteriormente, colaborem com outros agentes dentro de ambientes multiagentes.

4 AGENTES COM AGENTS SDK DA OPENAI

À medida que os agentes baseados em modelos de linguagem se tornam mais sofisticados, surge a necessidade de ferramentas que permitam estruturá-los de maneira robusta, modular e escalável. Criar agentes apenas com chamadas diretas a modelos pode funcionar em protótipos simples, mas rapidamente se torna difícil de manter quando o sistema precisa planejar, usar ferramentas, colaborar com outros agentes e operar em ciclos contínuos de decisão.

Foi justamente nesse contexto que surgiram os frameworks voltados para a construção de sistemas agentivos. O Agents SDK da OpenAI representa uma dessas abordagens modernas, projetado para facilitar o desenvolvimento de aplicações baseadas em agentes com um conjunto mínimo de abstrações, mas com alta flexibilidade arquitetural.

O princípio central do SDK é simples: um agente é essencialmente um modelo de linguagem configurado com instruções e ferramentas que lhe permitem agir no ambiente. Essa definição transforma o modelo de um sistema passivo em um executor de tarefas orientado por objetivos.

Dentro desse ecossistema, o agente deixa de ser apenas uma função que responde a prompts e passa a ser um componente operacional capaz de utilizar ferramentas, delegar tarefas e manter rastreabilidade de suas ações. Isso permite que desenvolvedores construam aplicações mais próximas de sistemas autônomos reais.

O SDK foi projetado para permitir que agentes compreendam objetivos, utilizem ferramentas externas, transfiram tarefas para outros agentes e operem dentro de fluxos mais complexos.

Podemos ilustrar esse conceito construindo um agente simples utilizando o Agents SDK em Python. A estrutura básica envolve definir o agente, suas instruções e as ferramentas disponíveis.


```

1 | !pip install openai-agents
2 |
3 | import asyncio
4 | from agents import Agent, Runner, function_tool
5 | from google.colab import userdata
6 | import os
7 |
8 |
9 | os.environ["OPENAI_API_KEY"] =
userdata.get('OPENAI_API_KEY')
10|
11| @function_tool
12| def calcular_desconto_func(preco: float) -> float:
13|     return preco * 0.9
14|
15| agente_precificacao = Agent(
16|     name="Agente de Precificação",
17|     instructions="Você analisa preços e sugere
ajustes.",
18|     tools=[calcular_desconto_func],
19|     model='gpt-4o-mini'
20| )
21| )
22|
23| # Wrap the agent execution in an async function
24| async def run_agent_async():
25|     resultado = await Runner.run(
26|         agente_precificacao,
27|         "O preço do produto é 100 reais, qual o novo
preço?"
28|     )
29|     print(resultado.final_output)
30|
31| await run_agent_async()

```

Código-fonte 3 - Criando agente com o Agents SDK
 Fonte: Elaborado pelo autor (2026)

Nesse exemplo, o agente não apenas interpreta a pergunta, mas pode utilizar a função `calcular_desconto` como uma ferramenta operacional. Isso demonstra uma das características centrais do SDK: a capacidade de integrar funções Python diretamente ao raciocínio do modelo.

Outro elemento importante é que os agentes podem delegar tarefas entre si. Em vez de tentar resolver tudo sozinho, um agente pode transferir parte do problema para outro agente especializado. Esse mecanismo é conhecido como *handoff* e constitui a base da colaboração multiagente dentro do SDK.

Podemos construir um pequeno exemplo com dois agentes colaborando.

```

1 | import asyncio
2 | from agents import Agent, Runner, handoff
3 | from google.colab import userdata
4 | import os
5 |
6 | analista_vendas = Agent(
7 |     name="AnalistaVendas",
8 |     instructions="Analise dados de vendas e identifique
padrões de demanda.",
9 |     model='gpt-4o-mini'
10| )
11|
12| estrategista = Agent(
13|     name="Estrategista",
14|     instructions="Sugira ações estratégicas com base em
análises de negócio.",
15|     model='gpt-4o-mini'
16| )
17|
18|
19| triage_agent = Agent(
20|     name="TriageAgent",
21|     instructions="""
22|     Você é um agente de triagem.
23|
24|     Se a tarefa envolver:
25|     - análise de vendas → envie para o AnalistaVendas
26|     - tomada de decisão estratégica → envie para o
Estrategista
27|     """,
28|     handoffs=[
29|         analista_vendas,
30|         handoff(estrategista)
31|     ],
32|     model='gpt-4o-mini'
33| )
34|
35|
36| async def run_agent():
37|     resultado = await Runner.run(
38|         triage_agent,
39|         "Baseado na análise: Demanda baixa detectada, o que
fazer?")
40|     print(resultado.final_output)
41|
42| await run_agent()

```

Código-fonte 4 - Colaboração entre agentes com o Agents SDK
 Fonte: Elaborado pelo autor (2026)

Nesse cenário, o primeiro agente produz uma análise e o segundo transforma essa análise em decisão. Esse tipo de fluxo permite criar pipelines inteligentes compostas por múltiplos níveis de raciocínio onde os agentes podem acionar outros agentes.

De forma geral, o Agents SDK fornece uma base estruturada para a criação de agentes capazes de raciocinar, planejar e agir em ambientes complexos. Ele permite que desenvolvedores construam sistemas multiagentes com colaboração explícita entre componentes, mantendo controle sobre segurança e validação.

5 ARQUITETURA E CONCEITOS BÁSICOS DO OPENAI AGENTS SDK

Nesta seção, exploramos a estrutura conceitual por trás do OpenAI Agents SDK. Nosso objetivo é entender o que compõe esse SDK internamente, quais são seus elementos essenciais, como funcionam juntos, quais decisões de desenho foram tomadas, para que o leitor ganhe visão de “por dentro” antes de programar agentes complexos. Em seguida, veremos como os ciclos de execução (loop), delegações, validações e estados são articulados.

5.1 Os Quatro Pilares Fundamentais

O OpenAI Agents SDK foi propositalmente desenhado com apenas alguns conceitos centrais, de modo a manter simplicidade e clareza. Os quatro primitivos principais são: Agent, Handoff, Guardrail e Session. Esta escolha arquitetural busca equilibrar poder e elegância, minimizando abstrações desnecessárias.

O **Agent** é o núcleo: cada agente encapsula um modelo de linguagem (LLM), instruções (prompt de sistema ou instruções ao modelo), ferramentas (functions ou APIs externas), handoffs (delegações possíveis para outros agentes) e guardrails (checagens de segurança ou validade).

O **Handoff** é uma forma de delegar controle: se um agente determina que outra tarefa está fora de seu escopo ou que outro agente especializado pode resolver melhor, pode “entregar o bastão” para outro agente. Isso permite distribuir responsabilidades e construir sistemas de múltiplos agentes cooperativos.

Guardrails funcionam como barreiras de segurança e consistência. Há guardrails de entrada (input) que checam e rejeitam casos indesejados ou maliciosos antes mesmo de o agente rodar com um modelo caro, e guardrails de saída (output) que verificam se o resultado final obedece a regras definidas (estruturas, limites, palavras proibidas etc.). Se um guardrail disparar, pode interromper ou redirecionar a execução.

Já a **Session** fornece o mecanismo de **memória e histórico**: permite que múltiplas chamadas ao agente compartilhem o estado conversacional, que o agente

“lembre” de trocas anteriores e mantenha coerência sem que o desenvolvedor tenha que gerenciar manualmente todo o histórico de prompts e respostas.

Além desses, outra funcionalidade intrínseca é o **tracing** (rastreamento de execução). A SDK já inclui instrumentação automática de eventos como chamadas ao modelo, invocações de ferramentas, handoffs, disparos de guardrails, o que permite depuração, monitoramento e visualização do fluxo do agente.

5.2 O Ciclo de Execução de um Agente (Loop Interno)

Para entender como tudo isso funciona em tempo de execução, vamos acompanhar o ciclo interno de um agente:

1. Você chama o agente com uma entrada (prompt ou query).
2. O agente invoca o modelo de linguagem para gerar uma resposta intermediária.
3. Se essa resposta contiver uma delegação (handoff), o controle passa para outro agente, ajustando o loop com novo agente + entrada.
4. Se a resposta indicar o uso de uma ferramenta, o SDK executa essa função (com argumentos extraídos do output do modelo).
5. Após a execução da ferramenta, o resultado interno é injetado de volta no fluxo do agente, que pode continuar gerando novas saídas ou finalizar com `final_output`.
6. Se a execução exceder um número máximo de turnos (`max_turns`), uma exceção é disparada (`MaxTurnsExceeded`).
7. Durante esse processo, os guardrails verificam a entrada e a saída, e podem interromper ou redirecionar o fluxo conforme regras definidas.

Esse loop permite que agentes sejam “interativos internamente”, com múltiplas iterações antes de entregar um resultado. E você, desenvolvedor, só se preocupa em definir as ferramentas, instruções e guardrails.

5.3 Delegações (Handoffs) como Componentes Nativos

Ao adicionar ferramentas a um agente, você pode simplesmente declarar funções Python decoradas com `@function_tool` ou usar classes de tool predefinidas. O agente entende que essas funções podem ser chamadas quando o modelo “decidir” invocar uma ação.

O Handoff, por sua vez, é considerado como uma ferramenta especializada. Ao configurar `agent.handoffs`, você pode passar agentes diretamente ou utilizar a função `handoff()`, que permite customizar o nome da ferramenta de delegação, filtros de entrada etc.

```
1 | from agents import Agent, handoff
2 |
3 | billing_agent = Agent(name="BillingAgent",
4 |   instructions="..." )
5 |
6 | triage_agent = Agent(
7 |   name="TriageAgent",
8 |   instructions="Você decide qual agente deve tratar a
9 |   solicitação.",
10 |   handoffs=[billing_agent, handoff(refund_agent)],
11 |   model='gpt-4o-mini'
```

Código-fonte 5 - Pseudo-código: Handoffs Agents SDK
Fonte: Elaborado pelo autor (2026)

Nesse cenário, se o prompt exige algo referente a reembolso, o agente de triagem pode delegar a RefundAgent via handoff.

5.4 Guardrails

Os guardrails operam em paralelo à execução principal do agente. No caso de guardrails de entrada, eles recebem a mesma entrada que o agente e aplicam uma função de checagem que retorna um objeto `GuardrailFunctionOutput` (com campos como `tripwire_triggered`). Se `tripwire_triggered` for verdadeiro, é lançada uma exceção (`InputGuardrailTripwireTriggered`) antes do agente rodar o modelo, poupando custo de computação.

Para guardrails de saída, após o agente gerar uma saída final, a função do guardrail roda sobre essa saída. Se disparar um tripwire, lança `OutputGuardrailTripwireTriggered` ou pode gerar um redirecionamento conforme a configuração.

Essa arquitetura permite colocar “camadas de segurança” no pipeline: evitar solicitações maliciosas ou saídas que violem políticas, sem complicar o desenvolvimento do agente em si.

5.5 Sessões e Histórico Contextual

Uma `Session` gerencia o histórico de interações com o agente. Ao usar sessões, não é necessário reenviar todo o diálogo anterior, o SDK armazena as mensagens e as insere automaticamente como contexto para chamadas subsequentes. Isso facilita a continuidade da conversa e mantém a coerência.

Além disso, sessões permitem configurar guardrails por sessão ou por execução (`run_config`), além de controlar variáveis de contexto e listeners customizados no modelo subjacente.

5.6 Observabilidade via Tracing

Por padrão, o Agents SDK já coleta traces de execução. Cada chamada ao modelo, invocação de ferramenta, delegação (`handoff`) e acionamento de guardrail é um evento que pode ser visualizado em painel de rastreamento. Isso permite depurar fluxos complexos, comparar diferentes versões de agentes e entender gargalos.

Você pode desativar o tracing globalmente (via variável de ambiente `OPENAI_AGENTS_DISABLE_TRACING=1`) ou para uma execução específica, ajustando o `RunConfig.tracing_disabled`.

Portanto, a arquitetura do OpenAI Agents SDK se sustenta sobre uma base clara: cada agente gera decisões por meio de loops internos (invocação de modelo e execução de ferramentas), pode delegar para outros agentes (`handoffs`), validar segurança via guardrails, manter estado usando sessões e oferecer rastreamento

completo do pipeline. A simplicidade intencional dos seus primitivos facilita o raciocínio, a manutenção e a escalabilidade das aplicações que se criam sobre ele.

EMASP

6 FERRAMENTAS (FUNCTION TOOLS)

Quando estamos construindo agentes com o OpenAI Agents SDK, muitas vezes queremos que o agente não apenas gere texto, mas atue sobre o mundo: buscar dados, executar cálculos, consultar APIs ou manipular arquivos. Essas ações são fornecidas por tools (ferramentas).

Nesta seção, vamos entender os tipos de ferramentas suportadas, como defini-las, como usá-las dentro do agente, e os desafios práticos envolvidos.

6.1 O que são Tools no Agents SDK

As ferramentas são componentes que o agente pode invocar como parte de seu ciclo de execução. A SDK da OpenAI reconhece essencialmente três categorias:

1. **Hosted tools:** ferramentas gerenciadas pela OpenAI (executadas nos servidores da OpenAI), como busca na web (WebSearchTool), busca de arquivos (FileSearchTool), execução de código em *sandbox* (CodeInterpreterTool) e outras. Essas ferramentas já vêm integradas e permitem ao agente pedir ações externas sem que você precise implementá-las.
2. **Function tools:** funções Python definidas no código do desenvolvedor, decoradas para serem expostas ao agente como ferramentas invocáveis. É aí que residem a maioria das integrações ao domínio da aplicação.
3. **Agents como tools:** um agente pode ser utilizado como ferramenta por outro agente, ou seja, você permite que o agente principal invoque outro agente como se fosse uma função. Isso permite orquestrações internas sofisticadas.

Essas abordagens permitem que o agente vá além do simples raciocínio linguístico: ele pode agir e colaborar com serviços externos. No entanto, é fundamental gerenciar entrada e saída de cada tool com clareza e segurança.

6.2 Definindo uma Function Tool

Uma function tool é uma função Python comum, que recebe parâmetros tipados e retorna um valor tipado. Para que o agente SDK reconheça essa função como tool, basta decorá-la com `@function_tool` (ou equivalente) e inseri-la na lista tools do agente.

Por exemplo, vamos definir uma função para consultar condições climáticas via API fictícia:

```
1 | from agents import Agent, Runner, function_tool
2 | import requests
3 | import asyncio
4 |
5 | @function_tool
6 | def consulta_clima(cidade: str) -> dict:
7 |     # Suponha que exista uma API REST local que retorna
8 |     # {"temperature": 25.3, "condition": "ensolarado"}
9 |     # Para este exemplo, vamos simular uma resposta.
10 |     if cidade.lower() == "fortaleza":
11 |         return {"temperature": 30.5, "condition":
12 |         "ensolarado"}
13 |     else:
14 |         return {"temperature": 20.0, "condition":
15 |         "desconhecido"}
16 |
17 | agent = Agent(
18 |     name="AgenteClimaAvancado",
19 |     instructions="Use a ferramenta
20 |     `consulta_clima(cidade)` para responder sobre clima de forma
21 |     precisa.",
22 |     tools=[consulta_clima],
23 |     model='gpt-4o-mini'
24 | )
25 |
26 | async def main():
27 |     result = await Runner.run(agent, "Como está o tempo
28 |     em Fortaleza hoje?")
29 |     print("Resposta:", result.final_output)
30 |
31 | await main()
```

Código-fonte 6 - Agente simples com tool customizada
Fonte: Elaborado pelo autor (2025)

Nesse exemplo, o agente pode decidir invocar `consulta_clima` se achar que essa ferramenta ajudará a responder à pergunta de forma mais precisa. O SDK

cuidará de converter a cadeia de chamadas corretamente (prompt → decisão de tool → execução da função → retorno ao modelo para finalização).

6.3 Uso de Hosted Tools

Além de suas próprias function tools, você pode tirar proveito das hosted tools pré-existentes da OpenAI. Por exemplo, o WebSearchTool permite que o agente faça buscas na internet, consultando notícias, artigos ou dados atualizados:

```
1 | from agents import Agent, Runner, WebSearchTool
2 | import asyncio
3 |
4 | agent = Agent(
5 |     name="AgentePesquisa",
6 |     instructions="Pesquise a internet usando
WebSearchTool quando necessário para dar respostas mais
atuais.",
7 |     tools=[WebSearchTool()],
8 |     model='gpt-4o-mini'
9 | )
10|
11| async def main():
12|     res = await Runner.run(agent, "Quem venceu a Copa do
Mundo de clubes em 2025 e quantos gols o artilheiro fez
nessa edição?")
13|     print("Resposta:", res.final_output)
14|
15| await main()
```

Código-fonte 7 - Agente simples com tool built-in de Web Search
Fonte: Elaborado pelo autor (2025)

Essa ferramenta interna já acompanha gestão de schemas e integração com o modelo, o que reduz o trabalho do desenvolvedor. Você também pode combinar hosted tools e function tools: o agente pode utilizar o WebSearchTool para buscar informações, passar ao modelo ou a sua própria ferramenta para tratar ou formatar esses dados.

6.4 Agentes como Ferramentas (Agents as Tools)

Em situações mais avançadas, pode ser útil que um agente invoque outro agente como se fosse uma ferramenta, por exemplo, um agente analista especializado

é chamado por um agente principal para realizar uma tarefa específica. Dentro do agente principal, você o trata como uma tool. Isso permite modularizar responsabilidades entre agentes.

6.5 Desafios e Boas Práticas

Embora a ideia de tools seja poderosa, há desafios práticos. Você deve decidir quais funções expor como ferramentas, evitar expor funções inseguras ou criticamente sensíveis, validar entradas e saídas rigorosamente, tratar exceções da função (o que fazer se a API falhar?), e planejar o fluxo de fallback se a tool for indisponível.

Além disso, é bom documentar bem cada tool (descrições, parâmetros esperados, restrições). O modelo interpreta essas descrições e decide quando invocar a ferramenta, quanto mais claras forem as descrições, melhores serão as decisões do agente.

Outra prática recomendada é limitar o número de tools por agente ou especializá-las, para que o agente não fique sobrecarregado com múltiplas opções confusas, o que pode gerar erro no seu pipeline devido a escolha incorreta de tools por parte do Agente.

7 HANDOFFS: DELEGANDO CONTROLE ENTRE AGENTES

Nesta seção, vamos explorar o mecanismo de handoff do OpenAI Agents SDK, um recurso que permite que um agente transfira o controle de uma tarefa para outro agente especializado. Isso é fundamental para arquitetar sistemas multi-agente com delegação limpa, onde cada agente assume a parte que melhor domina.

7.1 O conceito de handoff no SDK

Em essência, um handoff (delegação de tarefa) permite que um agente, ao identificar que uma requisição exige expertise diferente, delegue a execução a outro agente. Ao usar o SDK, os handoffs são representados como tools (ou ferramentas) que o modelo pode “invocar” para transferir a tarefa para outro agente. Dessa forma, o agente principal inclui em sua configuração a lista de agentes aos quais pode “passar a vez”.

A delegação é tratada como parte natural da lógica do agente, se o modelo decidir que outro agente é mais adequado para responder, ele chama a ferramenta correspondente (handoff), passando o contexto apropriado e deixando que o agente especializado continue a execução.

7.2 Como criar um handoff

Ao instanciar um agente, você pode incluir um parâmetro handoffs, cujo valor pode ser uma Lista de objetos do tipo Agent ou instâncias criadas via função handoff(). O SDK usa isso para gerar ferramentas de handoff automaticamente para o modelo chamar. Por exemplo:

```

1 | from agents import Agent, handoff
2 |
3 | billing_agent = Agent(name="BillingAgent",
4 |   instructions="Responda dúvidas de cobrança.", model='gpt-4o-
   mini')
5 |
6 | refund_agent = Agent(name="RefundAgent",
7 |   instructions="Cuide de pedidos de reembolso.", model='gpt-
   4o-mini')
8 |
9 | triage_agent = Agent(
10 |   name="TriageAgent",
11 |   instructions="Direcione questões de cobrança ou
   reembolso aos agentes apropriados.",
12 |   handoffs=[billing_agent, handoff(refund_agent)],
   model='gpt-4o-mini')

```

Código-fonte 8 - Agente com handoffs
 Fonte: Elaborado pelo autor (2026)

Nesse código, o agente triador (TriageAgent) poderá delegar parte da conversa ao `billing_agent` ou ao `refund_agent`, conforme o que o modelo julgar mais adequado. A função `handoff()` oferece opções adicionais, como definir nome da ferramenta de handoff, descrição, filtros de entrada, callbacks de execução etc.

7.3 Personalizações e controle de handoffs

Você pode customizar o comportamento do handoff de diversas maneiras. Um exemplo é utilizar argumentos como `tool_name_override` ou `tool_description_override` para personalizar como o handoff aparece no prompt ou nas ferramentas expostas. Também é possível definir um `on_handoff` call-back, uma função que será chamada no momento em que o handoff é invocado, permitindo executar lógica adicional (por exemplo, logging, preparação de dados, validações) antes de delegar efetivamente ao agente destino.

Outro ponto de controle importante são os *input filters* (filtros de entrada). Quando o handoff ocorre, todo o histórico de mensagens (ou parte dele) pode ser repassado ao agente designado. Se quiser ajustar o que esse agente verá como contexto inicial (por exemplo, removendo invocações de ferramentas anteriores ou filtrando prompts específicos), você pode usar `input_filter`. Isso ajuda a evitar que o

agente delegado receba ruído ou chamadas de ferramenta irrelevantes ao seu escopo de trabalho.

7.4 Casos de uso

7.4.1 Handoff de agente triador a especialistas de atendimento

Em um contexto de serviço ao cliente, imagine que temos um agente de front-desk que atua como primeiro contato e pode delegar a agentes especializados (suporte técnico, cobrança, marketing) conforme a necessidade. O front-desk agent pode incluir os agentes especializados na lista de handoffs, de modo que, quando um pedido chega e ultrapassa seu escopo, ele transfere a conversa ao especialista. Em seguida, esse especialista continua o diálogo com toda a informação relevante já passada pelo agente original.

7.5 Retorno para agente triador (handoff bidirecional)

Uma questão avançada é: e se eu quiser que o agente especializado termine sua tarefa e depois “retorne” ao agente triador para consolidar ou entregar a resposta final? Esse padrão gera debate na comunidade, pois nem sempre o SDK lida automaticamente com esse tipo de retorno. Algumas estratégias sugeridas incluem:

- Incluir o agente triador como opção de handoff nos agentes especialistas, de modo que o especialista possa “delegar de volta” ao triador.
- Projetar prompts nos agentes especialistas que orientem explicitamente que, ao finalizar, eles “passarão a resposta de volta ao triador para entrega ao usuário”.
- Em alguns casos, adicionar instrutores ou guardrails que forcem o agente especializado a usar um handoff de retorno ao triador.

7.6 Boas práticas

Ao utilizar handoffs, é crucial documentar bem nos prompts que o agente compreenda que pode delegar, usando prefixos ou instruções recomendadas disponíveis no SDK (exemplo: `RECOMMENDED_PROMPT_PREFIX`). Isso ajuda o modelo a entender que deve acionar o handoff como ferramenta e não simplesmente gerar texto narrativo.

Quanto mais agentes você incluir como handoffs, maior o risco de o modelo escolher equivocadamente delegar uma tarefa que poderia resolver diretamente. Por isso, cada agente deve ter responsabilidades bem delimitadas. Vale configurar guardrails de entrada para evitar que o agente triador tente delegar tarefas triviais.

Também é importante testar cenários de handoff múltiplo ou “encadeado” (onde o agente A delega para B, que delega para C).

8 GUARDRAILS: VALIDANDO ENTRADA E SAÍDA COM SEGURANÇA

Ao projetar agentes com o OpenAI Agents SDK, é crucial que eles não apenas executem tarefas, mas também o façam dentro de limites seguros e previstos. O mecanismo de guardrails (ou “guardas de segurança”) é uma camada que roda em paralelo ao agente propriamente dito, verificando tanto a entrada fornecida pelo usuário quanto a saída gerada, para garantir que o agente não se desvie de regras, execute operações inadequadas ou produza respostas fora do escopo desejado.

Essa seção explora em detalhes como usar guardrails, com exemplos práticos e explicações de seu funcionamento.

8.1 O papel dos Guardrails

Guardrails são definidos para atuar como barreiras de segurança: um input guardrail recebe a entrada do usuário antes que o agente a processe e pode interromper a execução caso a entrada viole alguma regra; um output guardrail verifica o resultado final do agente antes de entregá-lo ao usuário e pode rejeitar ou alterar a resposta se ela estiver fora das regras definidas.

As vantagens desse mecanismo são duas: primeiro, evitar que as entradas maliciosas ou irrelevantes custem recursos computacionais ao agente; segundo, prevenir que as respostas inadequadas, de conteúdo impróprio, vazamento de informação sensível, ataques de prompt injection, entre outras, cheguem ao usuário.

A execução do guardrail ocorre de forma “otimista”, ou seja, enquanto o agente continua seu processamento, o guardrail roda em paralelo e, se for acionado (quando a “tripwire” for disparada), interrompe ou cancela a execução do agente.

8.2 Funcionamento técnico de Guardrails

8.2.1 Input Guardrails

Quando um agente é chamado, antes de ele iniciar seu loop de processamento com o modelo de linguagem, os guardrails de entrada são avaliados. O SDK aplica cada função de input guardrail ao input e ao contexto do agente. Se algum deles retornar um resultado com `tripwire_triggered = True`, ocorre uma exceção `InputGuardrailTripwireTriggered` e a execução do agente é interrompida imediatamente.

O processo de validação de input segue estes passos: recebe-se o input do usuário, aplica-se a função do guardrail (que gera um objeto do tipo `GuardrailFunctionOutput`), verifica-se se o tripwire foi ativado; se sim, o agente não roda o modelo para esse input.

8.2.2 Output Guardrails

Depois que o agente alcança seu resultado final (ou seja, já executou ferramentas, handoffs, iterações internas etc.), os guardrails de saída entram em ação. Cada função de guardrail de saída recebe o output do agente e o contexto, e retorna um `GuardrailFunctionOutput`. Se `tripwire_triggered = True`, é lançada a exceção `OutputGuardrailTripwireTriggered`, interrompendo o envio desse resultado ao usuário.

Dessa forma, mesmo que o agente gere algo inesperado ou fora do escopo permitido, esse output pode ser interceptado e tratado antes de ser apresentado ao usuário final.

8.3 Guardrails em ação

A seguir, temos um exemplo em Python que demonstra como definir guardrails de entrada e saída. Imagine um agente de suporte ao cliente responsável por auxiliar

no cancelamento de assinaturas, que não deve aceitar mensagens com linguagem ofensiva nem retornar dados confidenciais.

```

1 | from agents import Agent, Runner, input_guardrail,
   output_guardrail
2 | from agents.guardrail import GuardrailFunctionOutput
3 | import asyncio
4 |
5 | # Guardrail de entrada: bloqueia inputs com palavrões
6 | @input_guardrail
7 | def block_bad_language(ctx, agent, user_input: str) ->
   GuardrailFunctionOutput:
8 |     lower = user_input.lower()
9 |     if "palavrão" in lower or "xingar" in lower:
10 |         return
   GuardrailFunctionOutput(output_info="linguagem ofensiva
detectada", tripwire_triggered=True)
11 |     return GuardrailFunctionOutput(output_info="input
limpo", tripwire_triggered=False)
12 |
13 | # Guardrail de saída: impede que o agente revele dados
sensíveis
14 | @output_guardrail
15 | def prevent_sensitive_output(ctx, agent, agent_output:
   str) -> GuardrailFunctionOutput:
16 |     # Exemplo simples: se aparecer "senha" ou "segredo",
   bloqueia
17 |     low = agent_output.lower()
18 |     if "senha" in low or "segredo" in low:
19 |         return
   GuardrailFunctionOutput(output_info="tentativa de vazamento
detectada", tripwire_triggered=True)
20 |     return GuardrailFunctionOutput(output_info="output
aceitável", tripwire_triggered=False)
21 |
22 | agent = Agent(
23 |     name="SupportAgent",
24 |     instructions="Você responde perguntas de suporte.",
25 |     model='gpt-4o-mini',
26 |     input_guardrails=[block_bad_language],
27 |     output_guardrails=[prevent_sensitive_output]
28 | )
29 |
30 | async def main():
31 |     try:
32 |         result = await Runner.run(agent, "Me diga sua
senha")
33 |         print("Resposta:", result.final_output)
34 |     except Exception as ex:
35 |         print("Guardrail ativado:", ex)

```

```
36 |  
37 | await main()  
38 |
```

Código-fonte 9 - Agente com guardrails

Fonte: Elaborado pelo autor (2026)

Nesse código, se o usuário pedir “Me diga sua senha”, o input guardrail identifica “senha” como termo sensível ou impróprio e interrompe a execução antes de o agente começar a processar a requisição. Se, por alguma razão, o agente ainda assim gerar uma resposta contendo “senha”, o guardrail de saída interceptará e bloqueará o resultado.

Esse modelo de uso ilustra como os guardrails funcionam como uma camada de segurança lógica sobre o agente.

8.4 Boas práticas

É recomendável usar guardrails para casos de jailbreak, prompt injection ou uso indevido do agente. Alguns guardrails podem, inclusive, ser agentes auxiliares, ou seja, a função do guardrail pode chamar outro agente para verificar a legitimidade da entrada ou da saída.

Outra prática importante é que os guardrails devem ser leves e eficientes, já que eles são executados em paralelo com o agente principal. Se forem muito custosos computacionalmente, poderão introduzir latência significativa.

8.5 Desafios e Limitações

Guardrails não substituem o design cuidadoso do agente. Eles são uma camada de proteção, mas se as instruções ou prompts não estiverem bem feitos, ainda pode haver comportamentos imprevistos.

Além disso, alguns casos muito complexos ou cheios de ambiguidade podem escapar das regras definidas. Por isso, é fundamental revisar os logs/traces e evoluir os guardrails conforme surgem casos reais.

Também é possível que guardrails acionem falsos positivos, isto é, rejeitem inputs ou outputs legítimos porque a regra foi muito restritiva. Por isso, durante a fase

de desenvolvimento, convém construir conjuntos de teste que exercitem casos-limite para calibrar bem os guardrails.

EMSE

9 SESSÕES E HISTÓRICO DE CONVERSAÇÃO

Ao trabalhar com agentes conversacionais, um dos maiores desafios é manter o contexto coerente ao longo de múltiplas interações. Não basta responder perguntas isoladas, espera-se que o agente “lembre” do que foi dito antes, ajuste respostas com base no passado, e aja de forma fluida. É exatamente aí que entram as Sessões (Sessions) no OpenAI Agents SDK: elas automatizam o gerenciamento do histórico de conversas, eliminando a necessidade de manipular manualmente prompts encadeados.

9.1 O que é uma Sessão no Agents SDK

Uma Session funciona como um objeto de memória que armazena todas as mensagens trocadas, entradas do usuário, respostas do agente, chamadas de ferramentas, delegações etc. Cada vez que você executa o agente com uma Session ativa, o SDK automaticamente recupera o histórico armazenado, o “prende” antes da nova entrada e, ao final da execução, grava todas as novas mensagens no histórico. Isso libera o desenvolvedor de concatenar manualmente prompts anteriores ou reconstruir contexto.

Esse comportamento reduz bastante o trabalho repetitivo de gerenciamento de contexto e torna os agentes mais naturais e consistentes.

9.1.1 Como funciona internamente

Quando você chama o agente com uma sessão:

1. Antes de rodar, a Session recupera o histórico anterior e prepara o "input list", que será enviado ao agente internamente.
2. Durante a execução, o agente pode gerar novas mensagens, seja do usuário, do agente, via ferramentas ou delegações, e esse conteúdo é capturado.
3. Ao final, o SDK grava todas as novas mensagens no histórico da Session, de forma que a próxima chamada começará com o contexto atualizado.

Esse ciclo se repete em cada execução com a mesma Session, mantendo a continuidade da conversa de forma transparente para quem programa.

9.1.2 Uso de Sessão

A seguir, temos um exemplo em Python ilustrando uma sessão simples com agente conversacional que lembra o contexto:

```
1 | from agents import Agent, Runner, SQLiteSession
2 | import asyncio
3 |
4 | agent = Agent(
5 |     name="AgenteMemória",
6 |     instructions="Você responde perguntas com contexto
do diálogo anterior.",
7 |     model='gpt-4o-mini'
8 | )
9 |
10| session = SQLiteSession("sessao_usuario_1")
11|
12| # Primeira interação
13| async def run_interactions():
14|     res1 = await Runner.run(agent, "Onde fica o Taj
Mahal?", session=session)
15|     print("Resposta 1:", res1.final_output)
16|
17|     # Segunda interação, sem re-enviar tudo manualmente
18|     res2 = await Runner.run(agent, "E em que país ele
está?", session=session)
19|     print("Resposta 2:", res2.final_output)
20|
21| await run_interactions()
```

Código-fonte 10 - Agente com memória
Fonte: Elaborado pelo autor (2025)

No primeiro turno de interação, o agente responde normalmente sobre a localização do Taj Mahal. No segundo, ele “lembra” que já falamos sobre Taj Mahal, e responde “Índia” sem que você precise reconstruir o contexto. Esse comportamento é automático graças à Session.

9.1.3 Gerenciamento e limitações do histórico de memória da Session

Embora manter todo o histórico seja conveniente, há limitações práticas. Em diálogos muito longos, o histórico acumulado pode ultrapassar o tamanho máximo de tokens que o modelo consegue “ver” (o context window). Para lidar com isso, o SDK oferece técnicas como trimming (remover partes antigas) ou compressão/sumário de histórico para manter apenas o essencial.

Outra técnica é usar `pop_item()` da Session, que permite “remover” ou corrigir a última mensagem gravada no histórico, útil em casos onde você quer desfazer uma resposta ou ajustar o diálogo.

9.1.4 Sessões em agentes de tempo real (RealtimeSession)

Além do modo tradicional, o SDK possui suporte a agentes em modo “realtime”, especialmente úteis para uso de áudio, streaming e interações contínuas. Nesse caso, a classe `RealtimeSession` gerencia não apenas histórico de mensagens, mas também conexões com o modelo em tempo real, recebimento e envio de eventos (mensagens de texto ou áudio) e integração com fluxos de interação contínua.

Dentro de uma `RealtimeSession`, você pode enviar mensagens utilizando `send_message()` ou `send_audio()` e iterar sobre eventos gerados (respostas, chamadas de ferramentas, delegações etc.). O histórico local é mantido e sincronizado com a sessão para garantir coesão.

9.1.5 Importância prática

Para alunos e desenvolvedores, a utilização de Sessions representa um grande salto de produtividade e usabilidade. Com ela, você pode focar mais na lógica do agente (ferramentas, prompts, delegações) e menos em gerenciar contexto. Além disso, ao combinar sessões com técnicas de trimming e compressão, é possível escalar agentes conversacionais com diálogos longos e realistas, sem estourar limites de tokens ou performance.

Essas abordagens são responsáveis por parte do que acontece em sistemas como chatbots comerciais ou assistentes virtuais que “lembram” conversas anteriores, tornando experiências mais naturais e sofisticadas.

EMEND

10 COMPARAÇÕES COM OUTROS FRAMEWORKS, LIMITAÇÕES E BOAS PRÁTICAS

Ao avançar no uso do OpenAI Agents SDK, é fundamental refletir criticamente sobre suas limitações, observar em quais cenários ele é mais apropriado, e adotar boas práticas para evitar armadilhas comuns. Nesta seção, compararemos o SDK com alternativas, discutiremos os limites técnicos atuais e apresentaremos recomendações para uso seguro e eficaz.

10.1 Comparações com outras abordagens e frameworks

O OpenAI Agents SDK propõe uma abordagem minimalista: ele oferece primitivas suficientes (agentes, ferramentas, handoffs, guardrails, sessões, tracing) para construir agentes robustos, mas sem impor camadas complexas de abstração. Segundo a documentação oficial, ele entrega o loop de agente, orquestração de ferramentas, delegações e observabilidade integradas.

Em contraste, frameworks mais maduros como LangChain ou agentes construídos “do zero” costumam oferecer enormes ecossistemas, memórias customizadas, conectores com bancos de dados, pipelines extensíveis, caches etc. Alguns de seus usuários comentam que, apesar da flexibilidade, a complexidade se torna um fardo em projetos maiores.

A vantagem do Agents SDK é justamente sua “barraca leve”: você pode adicionar exatamente aquilo que precisa sem carregar dependências extras. Um artigo técnico destaca que o SDK alivia parte da carga de orquestração, como decidir quando chamar ferramentas ou agentes, deixando o desenvolvedor focar na lógica de domínio.

Mas isso tem *trade-offs*. Com menos abstrações prontas, você pode ter que reimplementar, ou integrar externamente, partes que frameworks completos já fornecem. Por exemplo, mecanismos sofisticados de memória de longo prazo, caching de embeddings, roteiros de fallback complexos etc.

10.2 Limitações técnicas conhecidas

Uma das limitações frequentemente mencionadas é que nem todos os provedores de modelo de LLM suportam todas as funcionalidades. Por exemplo, alguns modelos não suportam **structured JSON outputs** ou não conseguem respeitar rigorosamente schemas JSON com ferramentas. A documentação alerta que, se você utilizar provedores que não suportam essas funcionalidades, pode haver falhas estruturais no agente.

Outro ponto sensível é o contexto (quantidade de histórico ou dados que você manda ao modelo). Passar muitos tokens de contexto pode exceder os limites do modelo ou causar erros de token limit exceeded. Alguns desenvolvedores recomendam fazer trimming (corte) ou sumarização do histórico antes de repassá-lo ao agente. Outra limitação importante é que ferramentas hosted (como WebSearchTool, FileSearchTool) são suportadas em modelos da OpenAI com a API adequada, mas nem sempre funcionam se você usar provedores que não tenham integração com essas ferramentas.

Por fim, um ponto de atenção é que, ao misturar modelos de diferentes provedores no mesmo fluxo, há risco de inconsistências, porque alguns modelos podem interpretar chamadas a ferramentas de modo diferente ou ter capacidades distintas de raciocínio.

10.3 Boas práticas de uso

Para minimizar riscos e extrair o melhor do SDK, é importante adotar boas práticas.

- Sempre limite o tamanho do contexto enviado ao modelo. Quando o histórico ficar muito grande, adote técnicas de resumo ou corte das interações mais antigas.
- Defina guardrails apropriados para entrada e saída, a fim de proteger contra inputs maliciosos ou saídas fora de escopo. Isso não só melhora a confiabilidade, mas também evita que seu sistema execute ações indesejadas.

- Teste seu agente com evals. Uma integração útil é com o Langfuse, que permite avaliar agentes criando conjuntos de casos de teste e verificar se o output é correto ou não, inclusive utilizando modelos como “juízes” para pontuar as saídas.
- Monitore suas execuções via tracing e registre métricas relevantes (tempo de chamada do modelo, tempo de execução de ferramentas, quantas delegações ocorreram etc.). Isso facilita a depuração e a identificação de gargalos.
- Evite incorporar dados sensíveis ou confidenciais diretamente no histórico ou nos prompts. Prefira abstrair ou anonimizar o contexto antes de passá-lo ao agente.
- Quando utilizar handoffs, documente bem no prompt ou nas instruções quais agentes podem assumir a tarefa e sob quais condições, para que o modelo tenha clareza de quando delegar ou não.
- Se for utilizar provedores que não suportam JSON estruturado, teste exaustivamente os outputs em cenários críticos e prepare fallback para quando o output vier mal formatado.
- Considere usar rate limiting, backoff retries e políticas de fallback nas chamadas de agentes, para lidar com erros transitórios da API.
- Finalmente, mantenha um repositório de casos de erro e observações (um FAQ interno), para que você possa evoluir o agente conforme ele enfrenta situações reais. Essa prática de “lições aprendidas” ajuda muito ao longo do tempo.

CONCLUSÃO

Este capítulo apresentou o OpenAI Agents SDK como uma biblioteca leve e orientada para Python, que permite construir agentes inteligentes com poucas abstrações, mas com poder expressivo. Conforme descrito na documentação oficial, o SDK opera sobre primitivas fundamentais: agentes equipados com instruções e acesso a ferramentas (tools), delegações entre agentes (handoffs), verificações de segurança de entrada e saída (guardrails), sessões automáticas de contexto (sessions) integrado para visualizar o fluxo de execução.

A partir desses pilares, desenvolvemos agentes básicos, aprendemos a associar tools (funções Python ou hosted tools) ao agente, configuramos delegações por handoffs, asseguramos segurança com guardrails e mantivemos contexto com sessions. Em cada etapa, exploramos exemplos práticos, destacamos limitações e boas práticas.

A principal vantagem do Agents SDK reside em sua simplicidade e coerência de design. Ao concentrar as responsabilidades em poucas primitivas bem definidas, o SDK permite que você escreva agentes sem depender de camadas profundas de abstração, mantendo controle direto sobre ferramentas, delegações e contexto. Essa filosofia facilita aprendizado, debug e manutenção.

Entretanto, esse desenho também traz certos desafios. Por depender de poucas abstrações, certas funcionalidades, como memórias de longo prazo sofisticadas, caching inteligente, adaptação dinâmica de ferramentas ou suporte nativo a muitos provedores de modelos de linguagem (LLMs), ficam a cargo do desenvolvedor. Em projetos mais ambiciosos, pode ser necessário estender ou complementar o SDK com soluções externas. Além disso, como muitos recursos (tools, guardrails, tracing) dependem da infraestrutura da OpenAI e do suporte do modelo, há restrições conforme o provedor adotado.

Você agora conhece os pilares do agente (loop, tools, handoffs, guardrails, sessions), sabe combinar esses elementos em fluxos efetivos e está preparado para enfrentar limitações e desafios do mundo real.

Mais do que aprender a usar uma SDK, você ganhou uma mentalidade de engenharia para agentes: pensar em delegação, segurança, observabilidade e robustez.

EMASP

REFERÊNCIAS

CEBALLOS, Cristieneil. What Are AI Agents? **Tutorials DOJO**, 2025. Disponível em: <<https://tutorialsdojo.com/what-are-ai-agents/>>. Acesso em: 26 fev. 2026.

GABRIEL, I. *et al.* The ethics of advanced AI assistants. [S. l.]: **arXiv**, 2024. Disponível em: <<https://doi.org/10.48550/arXiv.2404.16244>>. Acesso em: 26 fev. 2026.

GROOTENDORST, Maarten. A Visual Guide to LLM Agents. **Newsletter Maarten Grootendorst**, 2025. Disponível em: <<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-llm-agents>>. Acesso em: 26 fev. 2026.

MORINO, Sarah. Single Agent vs Multi-Agent AI: Why Multi-Agent Systems Are the Future of Automation. **Medium**, 2025. Disponível em: <<https://medium.com/@SarahMorino/single-agent-vs-multi-agent-ai-why-multi-agent-systems-are-the-future-of-automation-1f1a9a3fe42e>>. Acesso em: 26 fev. 2026.

OPENAI. **Documentação Agents SDK**. [S. l.], 2026. Disponível em: <<https://openai.github.io/openai-agents-python/>>. Acesso em: 26 fev. 2026.

OPENAI. **Documentação OpenAI Cookbook**. [S. l.], 2026. Disponível em: <<https://cookbook.openai.com/>>. Acesso em: 26 fev. 2026.

OPENAI. **Introducing Operator**. [S. l.], 2025. Disponível em: <<https://openai.com/index/introducing-operator/>>. Acesso em: 26 fev. 2026.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. [S. l.]: Pearson, 2021.

WOOLDRIDGE, M. **An Introduction to MultiAgent Systems**. 2. ed. [S. l.]: MIT Press, 2009.